Clawdy

> 一台手机就是全部：在 Android 手机本机运行的个人 Agent。 对话、工作区、命令和手机操作都在手机上完成；模型用你自己的 API Key，从手机直接连服务商。

- 版本：0.1.0-clawdy（开发中）
- 包名：`io.github.qigai.clawdy`
- 许可证：GPL-3.0-or-later（nanoMuse 衍生项目）
- 定位：个人使用，不上架应用商店

目录

1. 产品定位与边界
2. 核心能力
3. 总体架构
4. 模型接入（BYOK）
5. Termux 工作区
6. Hands：无障碍手机操作
7. Shizuku（可选）
8. 安全、隐私与审批
9. 界面设计规范
10. 界面清单与交互逻辑
11. 工程结构
12. 构建与安装
13. 首次配置（真机）
14. 验收标准
15. 开发路线与当前进度
16. 已知限制与风险
17. 许可证与合规
18. 设计评审流程
19. 决策记录
20. 图片与素材资源说明

---

1. 产品定位与边界

是什么 - 类似 nanoMuse 的 Android 本机 Agent。Agent 循环、工作目录、命令进程、文件和手机操作，全部由这台实体手机承担。 - 主要用途： - 用 Hands 自动操作手机上的其他 App。 - 在 Termux 后台处理 DOCX、PPTX、PDF，渲染网页，跑 Playwright/Chromium 测试，做 Node/Python 项目和 Git。

不是什么 - 不用云电脑、云手机、中继服务器，也不需要购买任何云资源。 - 不内置 Linux 沙箱（Alpine/PRoot）。命令执行统一交给 Termux（2026-10-15 决策）。 - 不内置本地模型。模型推理走用户自己的 API Key；本地模型以后作为可插拔选项再考虑。

参考项目与链接

项目	链接	用途
nanoMuse（fork 基础，GPL-3.0-or-later）	https://github.com/nano-muse/nanoMuse	Android 端 Agent 核心、Provider、Hands、Shizuku；fork 时的 HEAD 是 `a0d727f`（2026-10-15）
nanoMuse Android 源码	https://github.com/nano-muse/nanoMuse/tree/main/android/src/android	本项目 `src/clawdy` 直接来自这个目录
nanoMuse Hands 文档	https://github.com/nano-muse/nanoMuse/blob/main/docs/gui.md	无障碍读屏和手势的设计
nanoMuse 设备架构	https://github.com/nano-muse/nanoMuse/blob/main/docs/every-device.md	端侧运行时的整体思路
nanoMuse 自带 Key 说明	https://github.com/nano-muse/nanoMuse/blob/main/docs/own-key.md	BYOK 接入方式
nanoMuse Android 构建说明	https://github.com/nano-muse/nanoMuse/blob/main/android/BUILDING.md	构建环境、原生库
open-claude-ui（MIT，只作参考）	https://github.com/Caho1/open-claude-ui	设计令牌、输入框、消息排版、审批卡的结构参考
Termux	https://github.com/termux/termux-app	命令执行环境
Termux RUN_COMMAND	https://github.com/termux/termux-app/wiki/RUN_COMMAND-Intent	`termux_run` 用的官方接口
Shizuku	https://github.com/RikkaApps/Shizuku · https://shizuku.rikka.app/guide/setup/	可选的系统 API 能力
Android AccessibilityService	https://developer.android.com/reference/android/accessibilityservice/AccessibilityService	Hands 依赖的系统服务
Noto Serif SC / Source Serif 4	https://github.com/notofonts/noto-cjk · https://github.com/adobe-fonts/source-serif	内置的开源衬线字体（OFL）

两件事要分开看 - 「手机本地执行」：命令、文件、操作都在手机上，这一点已经做到。 - 「模型推理是否联网」：用 API Key 就要联网。任务文本，以及可能含个人信息的屏幕内容，会发送给你选择的服务商。App 会在首次使用时明确告知，并把上传的内容压到最少（见第 8 节）。

---

2. 核心能力

能力线	负责什么	实现方式
对话 Agent	理解任务、规划、调用工具、汇报结果	nanoMuse Agent 核心，Kotlin 协程循环
Termux 工作区	Linux 命令、项目目录、文档处理、网页渲染、测试	Termux 官方 `com.termux.RUN_COMMAND` Intent，工具名 `termux_run`
Hands	读取屏幕语义、点击、滑动、输入	Android AccessibilityService，节点优先、截图兜底
Shizuku（可选）	用户授权范围内的系统 API / ADB shell	Shizuku binder；不当作 root
记忆与会话	会话历史、长期记忆、上下文压缩	本地 SQLite（Room）

三条执行能力线职责互不重叠。任何一条不可用时，其余功能照常工作，并在「工具连接」页给出清晰的降级提示。

2.1 从 nanoMuse 怎样 fork

思路：nanoMuse 的 Android 端已经有完整的 Agent 循环、多家 Provider、无障碍 Hands、Shizuku、记忆和会话。所以不自研底层，而是**整体 fork 它的 Android 工程**，在上面做三件事：

1. **换壳**：用 Clawdy 自己的 Compose 界面替换首页、对话页和设置入口。
2. **换执行线**：去掉 PRoot/Alpine 沙箱，命令统一交给 Termux。
3. **换品牌**：包名、名字、主题、图标、头像、人设。

内部 Kotlin 包名 `com.openminis.app` 保持不变，以减少和上游的合并冲突；只改 `applicationId`，所以能和 nanoMuse 装在同一台手机上。

获取与同步上游

```
git clone https://github.com/nano-muse/nanoMuse src/nanoMuse        # 上游，HEAD a0d727f
cp -r src/nanoMuse/android/src/android src/clawdy                    # fork 出 Android 工程
cd src/clawdy && git init && git add -A && git commit -m "baseline from nanoMuse android"
# 以后同步上游：在 src/nanoMuse 里 git pull，再用 diff 或 cherry-pick 把需要的改动合进 src/clawdy
```

保留不动（直接复用）

模块	路径（`app/src/main/java/` 下）	作用
Provider 层	`com/openminis/app/provider/`（`LLMProvider.kt`、`ProviderFactory.kt`、`openai/`、`anthropic/`、`gemini/`、`openrouter/`）	各家模型的流式调用和工具调用
无障碍 Hands	`accessibility/` `MinisAccessibilityService.kt`、`NodeRegistry.kt`；`sandbox/`	读节点树、点击、滑动、输入

模块	路径（app/src/main/java/ 下）	作用
	offload/AccessibilityOffloadHandler.kt	
Shizuku	offload/ShizukuManager.kt、ShizukuBackend.kt；sandbox/offload/ShizukuOffloadHandler.kt	可选的系统 API
风险闸门	io/github/nanomuse/guard/RiskGate.kt	敏感操作识别与审批
记忆与会话	data/（Room）、tools/MemoryTools.kt	本地 SQLite
文件工具	tools/FileReadTool.kt、FileWriteTool.kt、FileEditTool.kt	读写工作区文件
浏览器	tools/BrowserUseTool.kt、browser/	App 内网页操作

修改

文件	改了什么
app/build.gradle.kts	applicationId = "io.github.qigai.clawdy"，应用名改为 Clawdy
ui/theme/Theme.kt、ChatColors.kt，新增 ClawdyTokens.kt	暖奶油加珊瑚配色、深色模式、圆角、动效令牌；全局衬线字体
agent/SoulStore.kt	Clawdy 人设和系统提示词：Termux 优先，敏感操作先问
tools/AgentTools.kt、ui/chat/ChatViewModel.kt	注册并分发 termux_run、phone_hands；termux_run 经过 RiskGate
ui/navigation/	启动页改为 ClawdyShell
res/mipmap-*、res/drawable-nodpi/nm_avatar_*	二次元白底图标和头像

新增

文件	作用
tools/TermuxTool.kt	termux_run：通过 RUN_COMMAND 执行命令，返回 stdout、stderr 和退出码
clawdy/HandsTool.kt	phone_hands：直接调用 Hands，不再经过沙箱 shell
clawdy/ClawdyPresets.kt	2026-10-15 的服务商和模型预置目录
clawdy/ToolStatus.kt	Termux、无障碍、Shizuku、权限的实时状态检测
clawdy/ClawdyFlags.kt	功能开关，例如 SANDBOX_IN_AGENT=false
ui/clawdy/（Shell、TaskCard、Turns、Sheets、Screens、Components、Icons）	15 屏前端壳、任务卡、SVG 图标、组件
res/font/、assets/licenses/	思源宋体子集、Source Serif 4 及其 OFL 许可
scripts/termux-bootstrap.sh	Termux 一次性初始化脚本

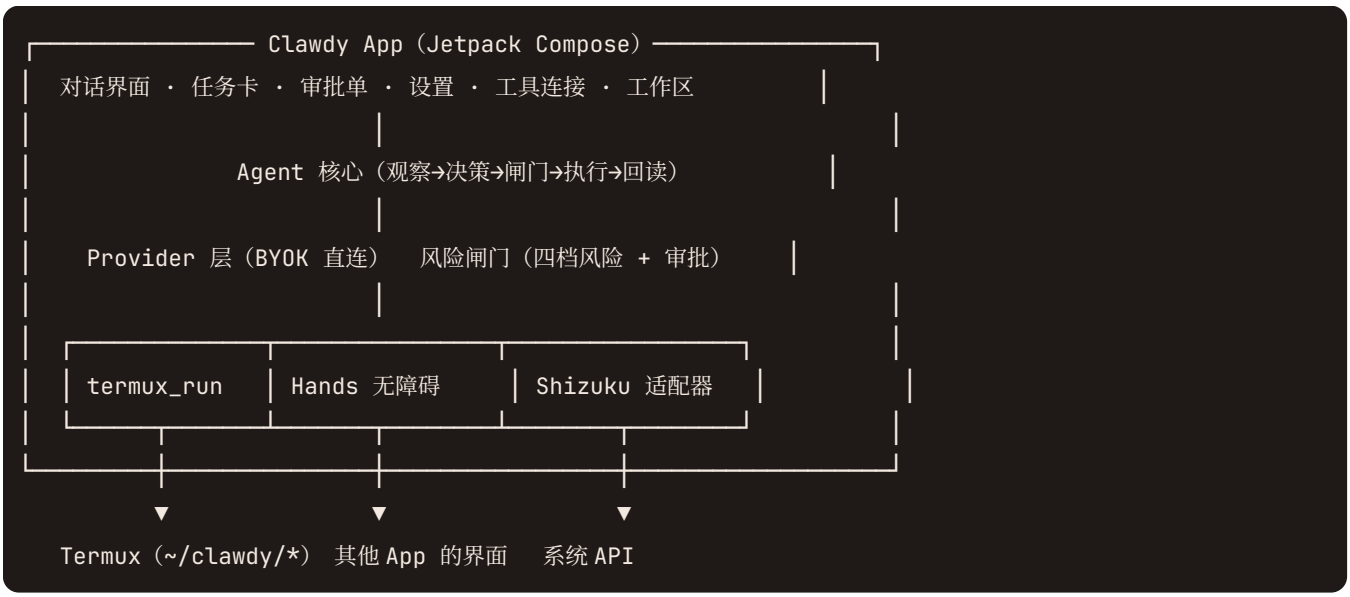
停用或移除

模块	处理方式
`sandbox/`（PRoot、Alpine rootfs、`shell_execute`）	从 Agent 工具列表和界面中移除，代码暂时保留以保证能编译
远程备份（rclone）	先 stub 掉
nanoMuse 的 `cloud/`、`web/`、`desktop` 等非 Android 部分	不 fork

原生库 `libpty_bridge.so` 等原生库直接取自 nanoMuse v1.0.5 正式版 APK（https://github.com/nano-muse/nanoMuse/releases ），不用 NDK 重新编译。

---

3. 总体架构



- 任务状态、审批、日志和停止信号都回到手机里的对话界面。
- 默认不同步任何文件、截图或会话到云端。远程备份（rclone）这一版暂时停用。

---

4. 模型接入（BYOK）

4.1 原则

- 用户自带 API Key，手机直接请求服务商，中间没有任何中转。
- Key 用 Android Keystore 加密后存在本机，不写日志，不进 Termux 环境，也不暴露给任何 WebView。
- 设置页只显示脱敏后的 Key（如 `sk-…a3f9`），并提供「测试连接」：返回延迟、是否支持工具调用；失败时给出具体原因（401 / 网络 / 证书）。

4.2 Provider 抽象层

- 统一接口：`chat(messages, tools, stream)`。

- 适配三种协议：
  - **OpenAI Chat Completions 兼容**：DeepSeek、通义千问（DashScope 兼容模式）、智谱 GLM、Kimi、OpenRouter、自建中转。Base URL 可以自定义。
  - **OpenAI Responses API**：GPT-6 系列的工具调用必须走 Responses，不能走 Chat Completions，需要单独适配。
  - **Anthropic Messages API**：Claude 系列，以及 DeepSeek 的 Anthropic 格式端点。
- 只支持带原生 tool calling 的模型；Termux、Hands、Shizuku 都注册成带 JSON Schema 的工具。
- 流式输出走 SSE。点「停止」时同时取消 HTTP 请求和正在执行的工具。
- 能拉取 `/models` 时自动刷新模型列表，拉不到就手填。

4.3 预置模型目录（截至 2026-10-15）

服务商	Base URL	模型 ID	备注
DeepSeek（默认）	`https://api.deepseek.com`	`deepseek-flash`	**DeepSeek V4.1 Flash，默认模型**；支持工具调用、识图、思考模式
DeepSeek	同上	`deepseek-v4-pro`	V4 Pro；支持工具调用和思考，不支持识图
OpenAI	`https://api.openai.com/v1`	`gpt-6-astra`、`gpt-6.1-sol`、`gpt-6-luna`	工具调用需要 Responses API
Anthropic	`https://api.anthropic.com`	`claude-opus-5-5`、`claude-sonnet-5-5`、`claude-haiku-5-5`	Messages API
Google Gemini	OpenAI 兼容端点	`gemini-3.8-flash`、`gemini-3.1-pro`	
通义千问	`https://dashscope.aliyuncs.com/compatible-mode/v1`	`qwen3.8-max`、`qwen3.8-flash`	
智谱 GLM	`https://open.bigmodel.cn/api/paas/v4`	`glm-5.3`、`glm-5.3-flash`	
Kimi	`https://api.moonshot.cn/v1`	`kimi-k3`	
OpenRouter	`https://openrouter.ai/api/v1`	`deepseek/deepseek-v4.1-flash` 等	列表从 `/models` 拉取
自定义 OpenAI 兼容	用户填写	示例：`gpt-6-astra`	用于中转或自建网关

> 模型更新很快。这张表是 2026-10-15 的快照，App 应优先用 `/models` 的实时结果。

4.4 成本控制

- 每个任务都有最大步数、最大 token 数和预算上限，界面实时显示用量。超出就暂停并询问。
- 「深度思考」开关只对支持推理的模型开放。

---

5. Termux 工作区

5.1 调用方式

- 通过 Termux 官方 `com.termux.RUN_COMMAND` 接口调用。命令、工作目录、环境变量都作为独立参数传入，不把用户输入拼成一条任意 shell 字符串。
- 每个任务有独立目录 `~/clawdy/<任务名>`，任务名只允许 `[A-Za-z0-9_-]`。
- 回调返回 stdout、stderr、退出码和任务 ID。超过 64 KB 的输出截断并提示。
- 支持超时、停止和重试；长任务用前台通知显示运行状态。

5.2 一次性初始化

- App 自带脚本 `scripts/termux-bootstrap.sh`（也打包在 APK assets 里），一次装好：
  - Python 3、Node.js、Git、Pandoc
  - python-docx、python-pptx、openpyxl、PDF 工具
  - Chromium、playwright-core
- 「工具连接」页可以随时「重新运行初始化」。

5.3 授权步骤（首次）

1. 安装 Termux。推荐 F-Droid 或 GitHub 版，不要用 Play 商店的旧版。
2. 在 Termux 里执行：

```
mkdir -p ~/.termux && echo 'allow-external-apps=true' >> ~/.termux/termux.properties
termux-reload-settings
```

3. 在系统设置里授予 Clawdy「在 Termux 中运行命令」权限（`com.termux.permission.RUN_COMMAND`）。
4. 未安装、权限被拒，或者进程被系统杀掉时，App 显示能看懂的修复提示，不会绕过授权。

5.4 隔离边界（如实说明）

- 目录隔离和命令白名单能减少误操作。但所有项目都在同一个 Termux 应用用户下运行，**这不是容器或虚拟机级别的隔离**。
- 运行来历不明的代码前，必须先展示要执行的命令和会改动的文件，取得同意后再运行。

---

6. Hands：无障碍手机操作

6.1 操作循环

观察 → 说明要做什么 → 执行一个动作 → 重新读取屏幕确认结果，每次只做一步。

- **节点优先**：先读无障碍节点树和按钮文字来定位，失败了才用截图坐标。
- **最少上传**：默认只把压缩后的节点文本发给模型。截图上传默认关闭；打开后也只在定位失败时才发。

- **看得见、能打断**：
  - 操作期间屏幕边缘有珊瑚色发光描边，即将点击的位置有脉冲圈。
  - 顶部悬浮胶囊显示当前动作，带暂停、继续、停止按钮。
  - 用户一碰屏幕就自动暂停 3 秒，避免抢操作。

6.2 不做的事

- 密码、PIN、验证码、CAPTCHA、卡号一律由用户亲手输入，Agent 不填写。默认不在密码框上做任何操作。
- 遇到 `FLAG_SECURE` 或其他受保护界面，不尝试绕过，只报告「不可读取」并交还给用户。
- 默认不在开机后自动继续跨应用操作。锁屏、权限被撤销或服务停止时立即暂停，恢复后重新确认状态再继续。

6.3 授权

- AccessibilityService 必须由用户在系统设置里亲手开启。「工具连接」页有无障碍权限引导：说明用途 → 跳转系统无障碍设置 → 返回 App 时自动检测是否已开启。
- 另外需要悬浮窗权限，用于发光描边和控制胶囊。

---

7. Shizuku（可选）

- 只提供用户授权范围内的系统 API 和 ADB shell 能力。不当作 root，也不假设所有 shell 命令都能用。
- Android 11 及以上，优先通过无线调试配对和启动；手机重启后要按系统限制重新启动。较旧的 Android 版本可能需要连接电脑。
- 未启动或能力不足时，无障碍和 Termux 功能照常工作，不要求提权。

---

8. 安全、隐私与审批

8.1 风险四档

档位	例子	处理方式
只读	读文件、看屏幕、查询	直接执行
普通写入	改工作区文件、生成文档	展示目标路径
执行命令	运行构建、安装依赖、跑脚本	展示完整命令
敏感操作	发送消息、发帖、支付、转账、删除、安装应用、改系统设置	**每一次都单独确认**，以前批准过也不能自动放行下一次

8.2 审批单

审批单包含： - 动作标题和对象。 - 将要发送或输入内容的预览，可以编辑。 - **本步会发给模型的数据**，例如「屏幕文字 12 条 · 截图 0 张」。 - 「本对话内同类操作不再询问」复选框：默认不勾；支付和密码类操作不提供这个选项。

App 在后台时，用高优先级通知和悬浮窗提醒你确认。

8.3 日志与恢复

- 每个任务在本地保存：步骤日志、命令和退出码、变更文件清单，以及必要的屏幕状态（文本形式）。
- 支持停止、重试单步、撤销 Git 改动、整体重跑。
- 运行日志可以在设置里查看和导出。

8.4 隐私默认值

- 截图上传：关。
- 不在密码框上操作：开。
- 文件、截图、会话默认只留在本机。

---

9. 界面设计规范

整体风格：暖白底、珊瑚橙强调色、衬线字体，软萌但克制。参考 Claude/Muse 的设计气质，但**不使用 Claude/Anthropic 的商标、Logo、专有字体或素材**，不做一模一样的界面。

9.1 颜色令牌

令牌	浅色	深色
背景 bg	`#FFFBF7`	`#1C1816`
表面 surface	`#FFFFFF`	`#2A2420`
正文 text	`#2B211C`	`#F3E9E1`
次要文字 text2	`#8A7A70`	`#A8988C`
描边 border	`#EFE3D9`	`#3A322C`
强调 accent	`#E07F5F`（按下 `#C9694B`）	`#F0A285`
任务卡	`#FDEBDD`，描边 `#F6D3BD`	`#3A2A22`
用户气泡	`#F9E1D6`	`#4A342A`
成功 / 警示 / 错误	`#4CAF7A` / `#E6A23C` / `#D9534F`	同色系提亮
Termux 紫	`#7B5CD6`	—
代码面板	`#1F1A17`，文字 `#F3E9E1`	同左

9.2 字体（评审反馈：中文尽量用 Anthropic Serif 这类风格）

- 中文：**思源宋体 Noto Serif SC**（OFL 开源），裁剪为常用字子集，内置 Regular 和 SemiBold 两个字重。
- 西文：**Source Serif 4**（OFL 开源）。
- 标题和正文都优先用衬线；代码用等宽字体。
- 不使用 Anthropic Serif 等专有字体，只取同类风格。

9.3 圆角、间距、动效

- 圆角：
  - 输入框 28dp，卡片 20dp，按钮 14dp，底部弹窗顶部 28dp，引用块 14dp。
  - 开关（评审反馈：圆角要小一点）：轨道 8dp 圆角矩形，滑块 6dp 圆角方块，不用全胶囊造型。
- 动效：

场景	动画
任务卡展开	260ms，`cubic-bezier(.2,.8,.2,1)`
底部弹窗	320ms
抽屉	280ms
开关滑块	160ms
按钮按下	缩放到 .92，120ms
时间线逐行出现	每行间隔 40ms
消息进入	上滑 12px 加淡入，200ms

9.4 图标（评审反馈）

- **全部用 SVG / 矢量图标，任何地方都不用 emoji**。
- 线条风格，描边 1.75。包括新对话、菜单、发送、停止、麦克风、附件、复制、重试、赞、踩、暂停、继续、关闭、工作区、定时任务、记忆、设置、终端、手、盾牌等。

9.5 角色与图标

- 应用图标和头像都用用户提供的二次元角色：白底，软萌风格。
- 头像有五种状态：

状态	表现
idle 待机	静止
working 工作中	呼吸动画
waiting 等待确认	冒汗、轻微晃动
happy 完成	弹跳一次
error 出错	变灰，加红点

---

10. 界面清单与交互逻辑

完整的逐页说明见 `design/Clawdy-交互说明.md`；可以点击的评审原型是「Clawdy 评审原型」Page。

编号	界面	要点
01	首页	头像加按时段变化的问候语；**没有建议按钮**；输入框上方只有一行「● 已连接工具 n/m」，点击进入工具连接；模型胶囊默

编号	界面	要点
		认 DeepSeek V4.1 Flash；新对话按钮是 SVG 图标
02	任务进行中	一轮里的工具调用和思考过程，收在一**张浅橙色任务卡**里，默认收起：第 n/N 步、当前动作、计时、进度条；运行时发送键变成停止键
03	任务卡展开	原位展开成步骤时间线；点某一步看详情
04	步骤详情	底部弹窗：命令、输出、错误三个标签页，exit 码徽章，复制输出，在 Termux 中打开
05	任务完成	卡片收成绿色「已完成 N 步」；正文回答；文件卡的「打开」**调用外部应用**（如 WPS）；复制、重试、赞、踩用 SVG 图标；失败时可以重试单步
06	敏感操作确认	琥珀色状态；消息预览用干净的引用块（左侧强调色竖条、浅底、14dp 圆角，不用粗边框）；显示本步发给模型的数据量
07	Hands 悬浮层	屏幕边缘发光描边、点击目标脉冲圈、顶部控制胶囊（SVG 暂停、继续、停止）、步骤提示
08	侧边栏	搜索、新对话、工作区文件、定时任务、记忆（SVG 图标）、最近会话（长按可重命名、置顶、删除）、底部 SVG 设置图标
09	模型选择	按服务商分组，带能力标签；深度思考开关；可添加服务商
10	添加服务商	选类型自动填 Base URL；Key 脱敏显示；默认模型下拉框用最新模型名；测试连接
11	工具连接	Termux、Hands 无障碍、Shizuku 三张状态卡（**没有 Linux 沙箱**）；**无障碍权限引导**；悬浮窗、通知、电池优化三项权限
12	首次引导	四步：欢迎 → 连接 Termux → 开启 Hands → 添加服务商；每一步都可以稍后再说
13	附件面板	拍照、相册、文件、当前屏幕、工作区、网页链接；联网搜索和 Hands 两个开关
14	工作区	Termux 下的 `~/clawdy/<任务>` 文件浏览；长按可分享、重命名、发给 Clawdy、删除（删除要二次确认）
15	设置	人设与提示词、模型、安全默认值、工具、运行日志、外观主题、关于（GPL-3.0 和第三方许可）

主流程

```
首次启动 → 引导(12) → 添加服务商(10) → 首页(01)
首页(01) →发送→ 任务卡收起(02) ⇄ 展开(03) → 步骤详情(04)
                  ↓ 敏感操作 → 审批单(06) → 继续
                  ↓ 需要操作其他 App → Hands 悬浮层(07) → 回到 Clawdy
                  ↓ 完成 → 结果与文件卡(05) → 用外部应用打开 / 工作区(14)
```

---

11. 工程结构

```
项目根/
├── src/nanoMuse/          上游源码（HEAD a0d727f）
├── src/open-claude-ui/    UI 参考（HEAD 2834bc1，MIT，仅参考）
├── src/clawdy/            Clawdy Android 工程（nanoMuse Android 的 fork）
│   ├── app/src/main/java/com/openminis/app/
│   │   ├── agent/SoulStore.kt          Clawdy 人设与系统提示词
│   │   ├── tools/TermuxTool.kt         termux_run 工具
│   │   ├── tools/AgentTools.kt         工具注册
│   │   ├── ui/chat/ChatViewModel.kt    工具分发与对话状态
│   │   └── ui/theme/                   Clawdy 设计令牌与主题
│   ├── app/src/main/res/               图标、头像、字体
│   └── scripts/termux-bootstrap.sh     Termux 初始化脚本
├── brand/                 图标与头像源文件
├── design/                概念图 s01-s15、交互说明、前端壳进度
└── docs/README.md         本文档
```

- 原生库（`libpty_bridge.so` 等）直接取自 nanoMuse v1.0.5 正式版 APK，不用 NDK 重新编译。
- Linux 沙箱已经从产品中移除：Agent 工具列表里不再有 `shell_execute`（PRoot），改为新增直接调用 Hands 的 `phone_hands` 工具；`termux_run` 走 RiskGate，命令有风险时会弹出 06 审批单。
- 前端壳代码：`ui/clawdy/`（ClawdyShell、ClawdyIcons、ClawdyComponents），设计令牌在 `ui/theme/ClawdyTokens.kt`。

---

12. 构建与安装

环境 - JDK 17、Android SDK（compile SDK 36、build-tools 35.0.0）、Gradle wrapper 8.11.1。 - 构建主机是 aarch64 时，x86_64 的 aapt2 要通过 qemu 运行。

命令

```
bash /workspace/tools/build.sh                # 等价于 ./gradlew --no-daemon :app:assembleDebug
-x lint
grep -nE "^e:|What went wrong|FAILED" /workspace/tools/build.log
ls -lh src/clawdy/app/build/outputs/apk/debug/
```

低内存主机注意 - 已经调好的参数：`-Xmx3584m`、`MaxMetaspaceSize=768m`、ParallelGC、`workers.max=1`，关闭并行和 daemon，KSP 非增量编译，Kotlin in-process。 - 主机没有 swap，overlay 文件系统也开不了 swapfile；按上面的参数可以稳定编译，一次完整编译约 9–10 分钟。

安装 - 用 `adb install -r app-debug.apk`，或者把 APK 传到手机上直接安装。 - 调试包用的是调试签名；以后换正式签名时，需要先卸载调试包。

---

13. 首次配置（真机）

1. 安装 Clawdy 和 Termux。
2. 在 Termux 里运行初始化脚本，并开启 `allow-external-apps`（见 5.3）。
3. 在系统设置里授予 RUN_COMMAND 权限。
4. 开启 Clawdy 的无障碍服务和悬浮窗权限；把 Clawdy 加入电池优化白名单。
5. （可选）启动 Shizuku 并给 Clawdy 授权。
6. 添加服务商，填入 API Key，点「测试连接」。默认模型是 DeepSeek V4.1 Flash。

---

14. 验收标准

- [ ] APK 能安装、启动，深浅色主题都符合设计令牌。
- [ ] 填入 Key 后能完成一次多步工具任务；断网或 Key 无效时有清晰提示。
- [ ] 在 Termux 中新建工作区并运行经过授权的命令，回传输出和退出码；长任务可以停止。
- [ ] 在 Termux 中完成 DOCX → PPTX、网页渲染截图、Playwright 测试各一次。
- [ ] Hands 每一步在屏幕上都看得见，可以随时暂停或停止；节点优先、截图兜底。
- [ ] 敏感操作每一次都弹出确认；密码、验证码、支付不由 Agent 填写。
- [ ] 关闭 Termux、无障碍或 Shizuku 后，功能降级清晰，有修复指引。
- [ ] 产出文件能用外部应用（如 WPS）打开。
- [ ] 用量和预算可见；截图上传可以关闭。
- [ ] 工作文件和日志默认只留在本机。
- [ ] 界面里没有 emoji 图标，中文使用衬线字体。

---

15. 开发路线与当前进度

阶段	内容	状态
0	克隆源码、许可证审计、构建环境	完成
1	品牌：包名、应用名、主题、图标、头像、人设	完成
2	`termux_run` 工具和初始化脚本	代码完成，待真机验证
3	设计：概念图、15 屏交互说明、可评审原型、两轮反馈	完成
4	真实 Compose 前端壳（按原型和反馈实现）：15 屏、设计令牌、衬线字体、SVG 图标、新开关、BYOK 模型目录、工具状态检测、无障碍引导	代码完成并提交，编译中
5	首个可安装的调试 APK	进行中（内存问题已解决，原版能编出 59.5 MB 的 APK）
6	Provider 层：Responses API（GPT-6）、模型目录、成本面板	待做
7	Hands 悬浮层、审批单与真实审批流对接	待做
8	Shizuku 状态检测与可选能力	待做
9	真机兼容性、权限、重启、后台限制、安全回归	待做
10	清理残留的 nanoMuse 文案和资源名；重新启用远程备份（rclone）	待做

---

16. 已知限制与风险

- **后台限制**：国产 ROM（如 OPPO ColorOS）会积极杀后台，必须把 Clawdy 和 Termux 都加入电池优化白名单，并允许自启动。
- **隐私**：使用云端模型时，任务文本和屏幕文字会发给服务商。已做最小化处理，但无法做到零上传。
- **Termux 不是沙箱**：见 5.4。
- **无障碍政策**：Android 把 AccessibilityService 定位为辅助功能；如果将来公开发布，需要单独审查商店政策和披露要求。目前只个人使用。
- **模型差异**：不同模型对工具调用的支持程度不一，GPT-6 必须走 Responses API。
- **远程备份**：rclone 暂时停用（原因：没有构建 `rclone.aar`）。

---

17. 许可证与合规

- `src/clawdy` 是 nanoMuse 的衍生作品，整体遵循 **GPL-3.0-or-later**：保留原版权声明、GPL 全文和第三方许可文件；分发时要提供对应源码。
- open-claude-ui（MIT）只作设计参考；如果直接使用了它的代码或资源，要保留 MIT notice。

- 字体 Noto Serif SC 和 Source Serif 4 遵循 SIL OFL 1.1，许可文本放在 `assets/licenses`。
- 不使用 Claude/Anthropic 的名称、Logo、专有字体或素材。界面中的「Anthropic 兼容」只是指协议类型。
- 二次元角色图片来自画师作品，**只限个人使用**；如果公开发布，必须先取得授权或更换素材。

---

18. 设计评审流程

1. 打开「Clawdy 评审原型」Page，切换到「检查」模式。
2. 点击任意组件，可以看到它的文字、字体、颜色、圆角和尺寸，还有它的交互和动画（可以▶重播）。
3. 给组件、交互或整页写建议，可以打上标签：改文字、改颜色、改字体、改尺寸、改动画、删掉、新增。
4. 点「导出反馈 txt」，下载文件或「复制全部」，发回给 Clawdy 开发对话。
5. 每一轮反馈会同步改进原型、Android 实现和本文档。

---

19. 决策记录

日期	决策
2026-10-15	模型改为用户自带 API Key 直连；本地模型不再是默认路线
2026-10-15	前端用 Compose 原生实现，不走 WebView；参考 Claude 设计令牌风格，但不照搬
2026-10-15	以 nanoMuse Android 工程（`a0d727f`）为基础整体 fork，接受 GPL-3.0；保留内部包名以便同步上游
2026-10-15	应用名 Clawdy，二次元白底软萌形象，暖奶油加珊瑚配色
2026-10-15	首页去掉建议按钮，只显示「已连接工具 n/m」
2026-10-15	工具步骤收进对话流里的浅橙色任务卡，点开才展开
2026-10-15	默认模型 DeepSeek V4.1 Flash（`deepseek-flash`）；模型目录更新到当日最新
2026-10-15	所有图标用 SVG，不用 emoji；开关圆角改小；中文统一用衬线字体
2026-10-15	去掉 Linux 沙箱（Alpine/PRoot），命令统一走 Termux
2026-10-15	11 页加入无障碍权限引导；产出文件用外部应用（如 WPS）打开
2026-10-15	自定义 OpenAI 兼容分组的示例模型改为 `gpt-6-astra`

---

20. 图片与素材资源说明

20.1 来源与授权

素材	来源	授权与限制
二次元角色（图标、头像）		

素材	来源	授权与限制
	用户提供的小红书画师作品；已抠掉黑底换成白底，并裁掉水印	**只限个人使用**；公开发布前必须取得画师授权或更换素材
15 张界面概念图	AI 图像模型生成	只用作设计参考，有生成瑕疵，不进 APK
评审原型 Page 中的界面	HTML/CSS 重画，头像复用上面的角色图	同角色图
图标集（58 个线条图标）	在 `ClawdyIcons.kt` 里自绘的 ImageVector，描边 1.75	本项目自有，GPL-3.0
字体	Noto Serif SC（裁成 GB2312 子集）、Source Serif 4、JetBrains Mono	SIL OFL 1.1，许可文本在 `assets/licenses/`
nanoMuse 原有的图片资源	上游仓库	GPL-3.0；不再使用的小龙动画已关闭

20.2 品牌源文件（`brand/`）

文件	尺寸	用途
`char_rgba.png`	1140×1140，透明底	角色主图；生成其他尺寸都以它为源
`icon_white_1140.png`	1140×1140，白底	高清应用图标母版
`clawdy_icon_512.png`	512×512	通用图标，用于 README 和分享
`clawdy_adaptive_fg_512.png`	512×512	自适应图标的前景层，四周留有安全边距
`avatar_preview.png`	512×512	头像效果预览
`preview.png`	—	图标效果预览

20.3 APK 内的资源（`src/clawdy/app/src/main/res/`）

路径	内容
`mipmap-*/ic_launcher*.png`（mdpi 到 xxxhdpi，含 night 目录）	白底应用图标，以及自适应图标的前景层（浅色版和深色版）
`mipmap-anydpi-v26/ic_launcher.xml` 等	自适应图标定义：白色背景 + 角色前景 + 单色层（Android 13 主题图标）
`drawable/ic_launcher_monochrome.xml`	单色主题图标
`drawable-nodpi/nm_avatar_{idle,working,waiting,happy,error}.webp`	对话头像。idle、working、waiting、happy 用同一张图，状态差异靠 Compose 动画实现（呼吸、冒汗晃动、弹跳）；error 是单独的一张图
`font/noto_serif_sc_{regular,semibold}.ttf`	中文衬线字体，每个约 3.4 MB
`font/source_serif_4_*.ttf`、`font/jetbrains_mono_*.ttf`	西文衬线字体和代码字体
`mipmap-anydpi-v26/ic_shortcut_*.xml`	桌面长按快捷方式：新对话、语音、拍照

20.4 设计稿（`design/`）

文件	内容
`s01_home.png` 到 `s15_settings.png`	15 屏概念图，768×1376，图上没有标注

文件	内容
`Clawdy-交互说明.md`	逐页交互逻辑，编号和概念图一一对应
`shell-progress.md`	前端壳实现进度
`check.png`、`contact0.png`、`contact1.png`	拼版检查用的中间文件，可以删除

20.5 以后要补的

- 每种头像状态各画一张表情图（目前只是同一张图配不同动画）。
- 启动页插画、空状态插画、出错插画。
- 如果公开发布，更换角色素材，或取得画师授权。